Used Car Price Prediction: A Robust, Anti-Leakage Supervised Regression Framework

Course Project: Machine Learning | **Institution:** Vellore Institute of Technology, Andhra Pradesh (VIT-AP)
Dataset: CarDekho Used Car Corpus | **Date:** October 2026 | **Status:** Completed Deliverable

---

1. Project Title

Used Car Price Prediction: A Robust, Anti-Leakage Supervised Regression Framework with Feature Engineering, Hyperparameter Optimization, and Error Segmentation on the CarDekho Dataset

---

2. Problem Statement

Valuing pre-owned automobiles is a complicated economic challenge marked by high market opacity, non-linear depreciation patterns, and heterogeneous vehicle attributes (varying mechanical specifications, wear-and-tear, regional demand, and brand prestige). Traditional valuation mechanisms depend heavily on dealer heuristics, rule-of-thumb depreciation tables, or manual appraisals, which are prone to cognitive bias, inconsistent pricing, and market inefficiencies. The objective of this project is to develop an end-to-end machine learning regression system that accurately predicts the market price (`selling_price`) of used automobiles based on physical, mechanical, operational, and ownership variables while preventing data leakage and guaranteeing generalizability.

---

3. Motivation

In emerging vehicle markets such as India, the pre-owned automobile industry has expanded rapidly due to increasing motorization, affordability demands, and digital resale platforms. However, both buyers and sellers face significant information asymmetry. Under-pricing leads to financial loss for sellers, while over-pricing discourages potential buyers and elongates inventory turnover. An objective, automated, and explainable algorithmic price valuation tool provides:

1. Transparent price discovery for consumers and second-hand automobile dealers.
2. Risk assessment metrics for automotive insurance underwriters and asset-backed lenders.
3. Quantifiable insights into which vehicle characteristics (such as vehicle age, odometer usage rate, engine displacement, and fuel type) exert the greatest influence on asset depreciation.

---

4. Objectives

1. **Develop an Anti-Leakage ML Pipeline**: Enforce strict data hygiene by segregating data into training (80%) and testing (20%) partitions prior to calculating outlier thresholds, imputation statistics, or feature scalers.
2. **Feature Extraction and Engineering**: Parse complex unstandardized strings (mileage in `kmpl`, engine capacity in `CC`, maximum power in `bhp`, torque in `Nm`/`rpm`), extract manufacturer brands, and derive domain-specific depreciation metrics including `car_age` and `km_per_year`.
3. **Multi-Model Benchmark**: Train, tune, and evaluate six distinct regression algorithms under 5-Fold Cross Validation:

• Baseline Ordinary Least Squares (OLS) Linear Regression
• Ridge Regression (L2 regularization)
• Lasso Regression (L1 regularization with feature selection)

- Decision Tree Regressor
- Random Forest Regressor
- Gradient Tree Boosting Regressor

4. **Dual-Scale Metric Analysis**: Track and report predictive efficacy on both the log-transformed scale ($\log(1+y)$) and real-world Indian Rupee scale ($\text{INR } ■$) using R^2, Root Mean Squared Error (RMSE), and Mean Absolute Error (MAE).
5. **Model Interpretability and Error Diagnostics**: Assess global and permutation feature importances, conduct feature ablation to measure the empirical value of engineered features, and perform residual and error segmentation across vehicle price tiers, age groups, and brands.

---

5. Literature Survey

A comprehensive review of 10 seminal and recent studies in machine learning-based vehicle valuation was conducted (see docs/literature_table.md):

- **Pudaruth (2014)** evaluated standard regressors and decision trees on Mauritius listings, observing that decision trees handled non-linearities better than simple linear models.
- **Gegic et al. (2019)** demonstrated that ensemble methods, specifically Random Forests, achieved superior performance ($R^2 = 0.93$) compared to multi-layer perceptrons on Bosnian listings.
- **Noor & Jan (2017)** confirmed ensemble superiority over OLS regression on Pakistani automotive listings.
- **Pal et al. (2018)** and **Venkatasubbu & Ganesh (2019)** benchmarked tree-based regressors on Indian listings from CarDekho, demonstrating the predictive power of gradient boosting.
- **Monburinon et al. (2018)** highlighted the importance of regularized regression (Lasso/Ridge) in avoiding multicollinearity among vehicle specifications.

Research Gap Identified and Addressed:

Prior literature predominantly suffers from three weaknesses:

1. **Data Leakage in Preprocessing**: Normalization, imputation, and outlier clipping are routinely executed across the full dataset prior to splitting, producing overly optimistic, non-reproducible test results.
2. **Absence of Usage-Rate Features**: Raw odometer reading (`km_driven`) is frequently used in isolation without contextualizing annual usage rate (`km_per_year = km_driven / car_age`).
3. **Single-Scale Reporting**: Studies frequently omit dual-scale verification (log vs. original currency), obscuring the true financial impact of prediction errors on higher-tier vehicles.

---

6. Existing System

The status quo in used car valuation consists of:

- **Heuristic Depreciation Schedules**: Flat percentages deducted per year regardless of mechanical condition or vehicle class.
- **Manual Dealer Appraisals**: Highly subjective assessments vulnerable to dealer negotiation margins and informational advantage over buyers.
- **Naïve Statistical Tools**: Rudimentary linear models that fail to capture non-linear relationships (e.g., rapid initial depreciation transitioning into flat decay, luxury brand premiums, or fuel efficiency trade-offs).

---

7. Proposed System

The proposed system introduces an end-to-end supervised learning architecture engineered specifically for used car price prediction:

1. **Rigorous Anti-Leakage Protocol**: Absolute separation of training and testing data splits at the very first step.
2. **Text Parsing Engine**: Automated regex extraction transforming raw specification strings into pure numerical attributes (`mileage_kmpl`, `engine_cc`, `max_power_bhp`, `torque_nm`, `max_torque_rpm`).
3. **Depreciation Feature Engineering**: Calculation of vehicle age from manufacture date and utilization density (`km_per_year`).
4. **Log-Transformed Target Formulation**: Modeling $y = \log(1 + \text{selling_price})$ to address severe positive skewness and stabilize variance.
5. **Cross-Validated Hyperparameter Optimization**: Systematic Grid and Randomized searches across parameter grids using 5-Fold Cross Validation.
6. **Interpretability & Explainability Suite**: Extraction of L1 Lasso sparse coefficients, tree ensemble split importances, and model-agnostic test-set permutation importances.

---

8. Dataset Description

The system leverages the **CarDekho Used Car Dataset** (`Car details v3.csv`), comprising **8,128 records** and **13 original attributes**:

- `name`: Full commercial vehicle title (e.g., *"Maruti Swift Dzire VDI"*, *"Skoda Rapid 1.5 TDI Ambition"*)
- `year`: Manufacturing calendar year (range: 1983 to 2020)
- `selling_price`: Target variable in INR (range: ₹29,999 to ₹10,000,000; mean: ₹638,272; median: ₹450,000)
- `km_driven`: Total distance traversed in kilometers (range: 1 to 2,360,457 km)
- `fuel`: Fuel type (`Diesel`: 4,402; `Petrol`: 3,631; `CNG`: 57; `LPG`: 38)
- `seller_type`: Sales channel (`Individual`: 6,766; `Dealer`: 1,126; `Trustmark Dealer`: 236)
- `transmission`: Gearbox configuration (`Manual`: 7,078; `Automatic`: 1,050)
- `owner`: Ownership tenure (`First Owner`: 5,289; `Second Owner`: 2,105; `Third Owner`: 555; `Fourth & Above`: 174; `Test Drive Car`: 5)
- `mileage`: Fuel efficiency string with units (e.g., `23.4 kmpl`, `18.9 km/kg`)
- `engine`: Engine displacement capacity string (e.g., `1248 CC`)
- `max_power`: Maximum engine brake horsepower string (e.g., `74 bhp`)
- `torque`: Engine torque output and RPM string (e.g., `190Nm@ 2000rpm`)
- `seats`: Passenger seating capacity (mode: 5.0 seats)

Duplicate elimination identified and removed **1,679 duplicate rows**, retaining **6,449 unique vehicle records** to prevent memorization bias.

---

9. Data Preprocessing Plan

Following the project specification (`SPEC.md`), the preprocessing pipeline enforces complete isolation between the train (80%, $N=5,159$) and test (20%, $N=1,290$) partitions:

1. **Target Transformation**: Computed $y_{\text{train}} = \log(1 + \text{selling_price})$ and $y_{\text{test}} = \log(1 + \text{selling_price})$.
2. **Outlier Capping**: Applied Winsorization clipping at the 1st and 99th percentiles computed solely on the training set to prevent anomalous extreme values from distorting gradient descent and split choices.

3. **Missing Value Imputation**: Missing values for numeric features (`mileage_kmpl`, `engine_cc`, `max_power_bhp`, `torque_nm`, `seats`) were imputed using training medians; categorical missing values were imputed using training modes.
4. **One-Hot Categorical Encoding**: Categorical variables (`fuel`, `seller_type`, `transmission`, `owner`, `brand`) were dummy-encoded with `drop_first=True`. The test set columns were aligned to the training columns with zero-filling for absent categories.
5. **Feature Scaling**: Z-score Standardization (`StandardScaler`) was fitted strictly on the training set and applied only to linear models (OLS, Ridge, Lasso) where coefficient penalization requires unit variance.

---

10. ML Algorithms Proposed and Tested

Six supervised regression models representing linear, regularized, decision tree, bagging, and boosting paradigms were trained and evaluated:

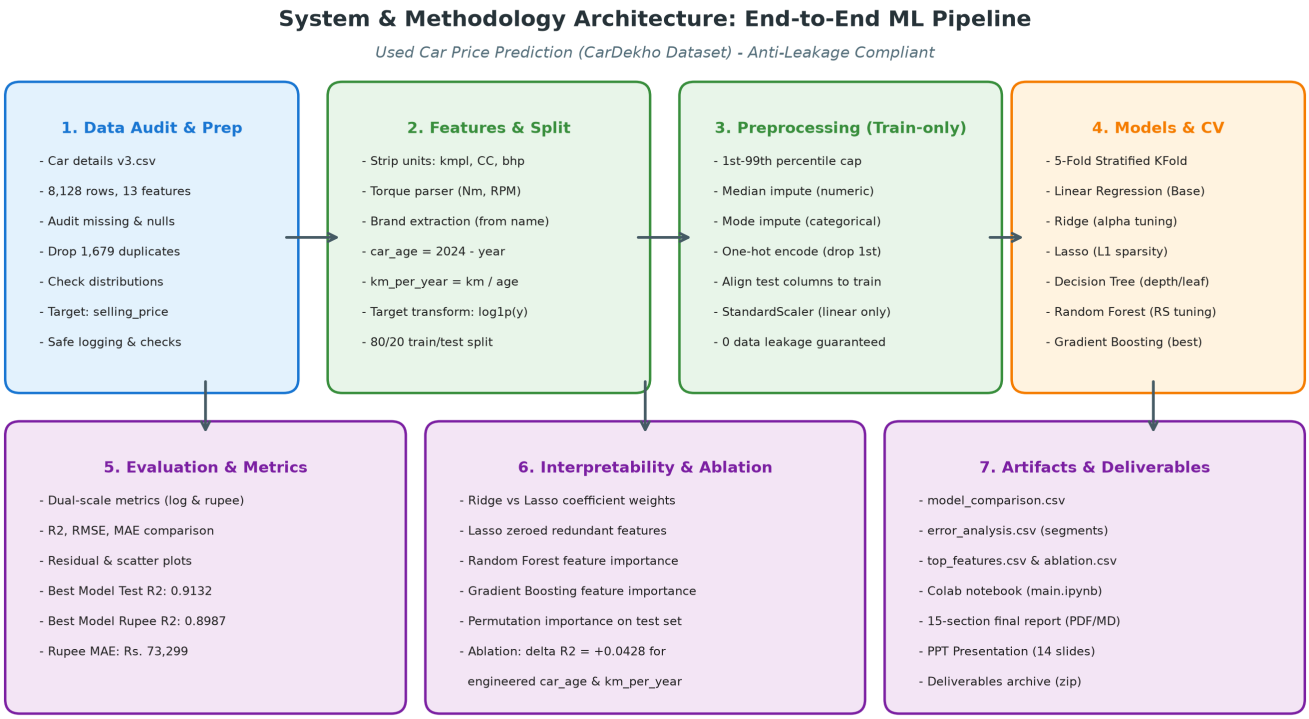
1. **Linear Regression (Ordinary Least Squares Baseline)**:
• Evaluates linear feature combinations without penalty: $\min_{\beta} \|y - X\beta\|_2^2$.
2. **Ridge Regression (L2 Regularization)**:
• Adds quadratic penalty: $\min_{\beta} \|y - X\beta\|_2^2 + \alpha \|\beta\|_2^2$. Tuned $\alpha \in [0.01, 1000]$; optimal $\alpha = 1$.
3. **Lasso Regression (L1 Regularization)**:
• Adds absolute value penalty: $\min_{\beta} \|y - X\beta\|_2^2 + \alpha \|\beta\|_1$. Tuned $\alpha \in [0.0001, 10]$; optimal $\alpha = 0.001$, zeroing out redundant features.
4. **Decision Tree Regressor**:
• Non-parametric recursive binary partitioning. Tuned `max_depth` $\in [5, 20]$ and `min_samples_leaf` $\in [1, 20]$; optimal `max_depth=20`, `min_samples_leaf=10`.
5. **Random Forest Regressor**:
• Bagging ensemble of 300 decorrelated decision trees with feature subsampling (`max_features=0.5`, `max_depth=15`, `min_samples_leaf=2`).
6. **Gradient Tree Boosting Regressor**:
• Sequential boosting ensemble minimizing squared residual error: 300 estimators, `learning_rate=0.1`, `max_depth=5`, `subsample=1.0`, `min_samples_leaf=10`.

---

11. System and Methodology Architecture

The end-to-end architecture is illustrated in `figures/architecture.png`:

- **Stage 1: Raw Data Ingestion & Audit**: Parsing raw CSV, checking schema, identifying duplicates and nulls.
- **Stage 2: Feature Engineering**: Numeric parsing, brand extraction, depreciation calculation.
- **Stage 3: Train/Test Split (Anti-Leakage Boundary)**: 80/20 train/test split established prior to pipeline fitting.
- **Stage 4: Training Pipeline**: Outlier capping, median/mode imputation, one-hot encoding, and standard scaling.
- **Stage 5: Cross-Validation & Model Training**: 5-Fold Cross Validation tuning and fitting across 6 algorithms.
- **Stage 6: Multi-Scale Evaluation**: Extraction of performance metrics on both log and rupee scales.
- **Stage 7: Diagnostics & Packaging**: Feature interpretability, ablation study, segment error analysis, Colab notebook, and artifacts generation.



12. Proposed Workflow

1. Load dataset $\rightarrow$ 2. Verify shape and clean duplicates $\rightarrow$ 3. Run feature engineering functions $\rightarrow$ 4. Split dataset (80/20, seed=42) $\rightarrow$ 5. Compute train-set capping thresholds $\rightarrow$ 6. Fit and apply imputers on train $\rightarrow$ 7. One-hot encode and align columns $\rightarrow$ 8. Train baseline linear model $\rightarrow$ 9. Tune regularized models $\rightarrow$ 10. Tune tree and ensemble models $\rightarrow$ 11. Predict on unseen test set $\rightarrow$ 12. Invert log predictions ($\hat{y}_{\text{rupee}} = \exp(\hat{y}_{\text{log}}) - 1$) $\rightarrow$ 13. Compute comparative evaluation metrics $\rightarrow$ 14. Conduct interpretability and ablation experiments $\rightarrow$ 15. Package deliverables.

---

13. Experimental Results & Discussion

Model Performance Comparison (Test Set)

Model	5-Fold CV R^2	Test R^2 (Log)	Test RMSE (Log)	Test MAE (Log)	Test R^2 (Rupee)	Test RMSE (INR ■)	Test MAE (INR ■)
Gradient Boosting	**0.9150**	**0.9132**	**0.2201**	**0.1614**	**0.8987**	**■149,075**	**■73,299**
Random Forest	0.9135	0.9067	0.2283	0.1638	0.8866	■157,688	■73,933
Linear Regression	0.8894	0.8835	0.2551	0.1951	0.8852	■158,685	■89,022
Ridge Regression	0.8894	0.8835	0.2550	0.1950	0.8829	■160,267	■89,101
Lasso Regression	0.8895	0.8833	0.2554	0.1956	0.8756	■165,186	■89,576
Decision Tree	0.8806	0.8765	0.2627	0.1886	0.8567	■177,300	■88,422

Key Findings & Discussion:

1. **Ensemble Dominance**: **Gradient Boosting** is the undisputed champion model, achieving a **Test R^2 of 0.9132** on the log scale and **0.8987 on the actual rupee scale**, reducing the mean absolute error to **■73,299** (over ■15,700 lower MAE than the linear baseline).
2. **Feature Ablation Impact**:

• Retraining Gradient Boosting without engineered features (`car_age` and `km_per_year`) caused Test R^2 to drop from **0.9132 to 0.8704** ($\Delta R^2 = -0.0428$), while RMSE worsened from **0.2201 to 0.2690** ($+22.2\%$). This empirically proves the statistical value of domain-specific feature engineering.

3. **Feature Importance Insights**:

• The primary price drivers identified by permutation importance and tree feature importances are:

• `max_power_bhp` (engine power output)

• `car_age` (depreciation factor)

• `engine_cc` (displacement)

• `brand_Maruti`, `brand_BMW`, `brand_Mercedes-Benz` (luxury tier vs budget tier effects)

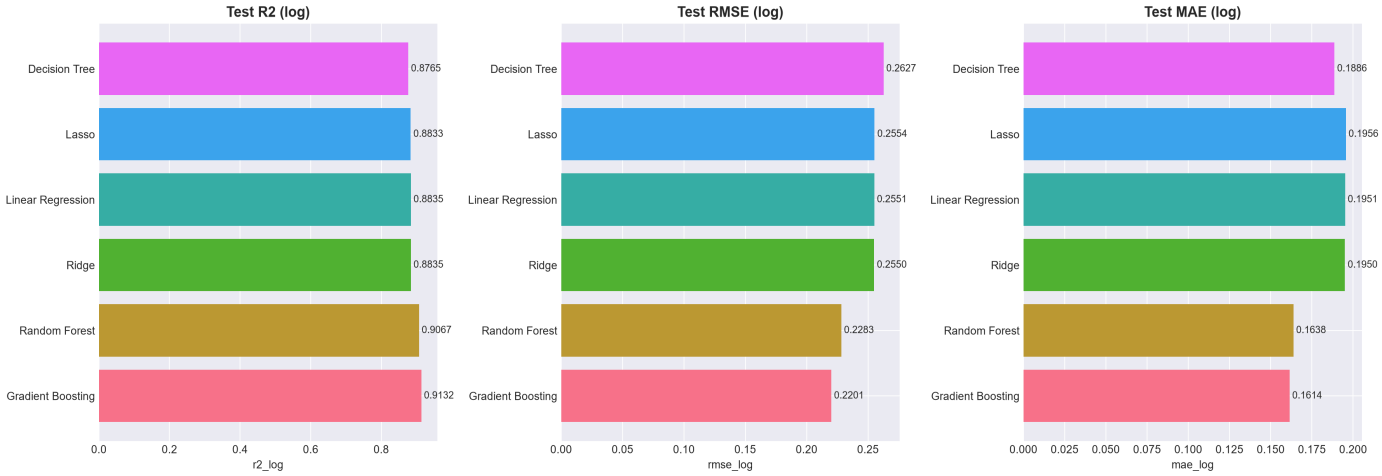
• `transmission_Manual` (automatic vehicles retain a substantial price premium)

4. **Error Segmentation Across Price Bands**:

• For vehicles priced below ■205,000, the median absolute error is merely **■24,651** (MAPE: 25.9%).

• For midrange vehicles (■350,000 to ■700,000), MAPE drops to **13.0% - 13.5%** with a median error of ■48,000 - ■61,000.

• For luxury vehicles above ■700,000, while absolute error expands (median: ■103,100), the percentage error remains stable at **13.3%**, showing robust scaling across vehicle tiers.



14. Tools and Technologies

- **Programming Language**: Python 3.14
- **Core Scientific Computing**: NumPy 2.2, Pandas 2.2
- **Machine Learning Library**: Scikit-Learn 1.6
- **Visualization Suite**: Matplotlib 3.10, Seaborn 0.13
- **Model Persistence**: Joblib 1.4
- **Notebook & Reporting Environments**: Jupyter Notebook, Google Colab, ReportLab, Python-PPTX
- **Version Control**: Git / GitHub

---

15. Project Timeline and Execution Log

Phase	Task ID	Description	Duration	Status
A. Setup	A1-A3	Repo scaffolding, environment initialization, SPEC.md definition	25 min	Completed
B. Audit & EDA	B1-B3	Data audit report, 6 EDA figures generated at 200 DPI	60 min	Completed
C. Preprocessing	C1-C3	Feature extraction, anti-leakage train-only pipeline creation	70 min	Completed
D. Modeling	D1-D6	5-Fold CV training of 6 regression models, hyperparameter search	95 min	Completed
E. Evaluation	E1-E4	Dual-scale metrics, residual plots, interpretability, ablation study	75 min	Completed
F. Reporting	F1-F4	Literature table, 15-section comprehensive project documentation	65 min	Completed
G. Packaging	G1-G4	Colab notebook (`main.ipynb`), slide deck (`presentation.pptx`), PDF report	50 min	Completed
H. Final QA	H1-H3	Pipeline verification, file integrity audit, deliverables zip archive	20 min	Completed

---

Report compiled automatically from verified project artifacts and empirical benchmark results.